

Point Of View

Enabling Observability for Data Pipelines

3rd Dec, 7:30- 9:00 pm IST

Join us to understand how Observability can enable a data analytics pipeline to realize its true potential.



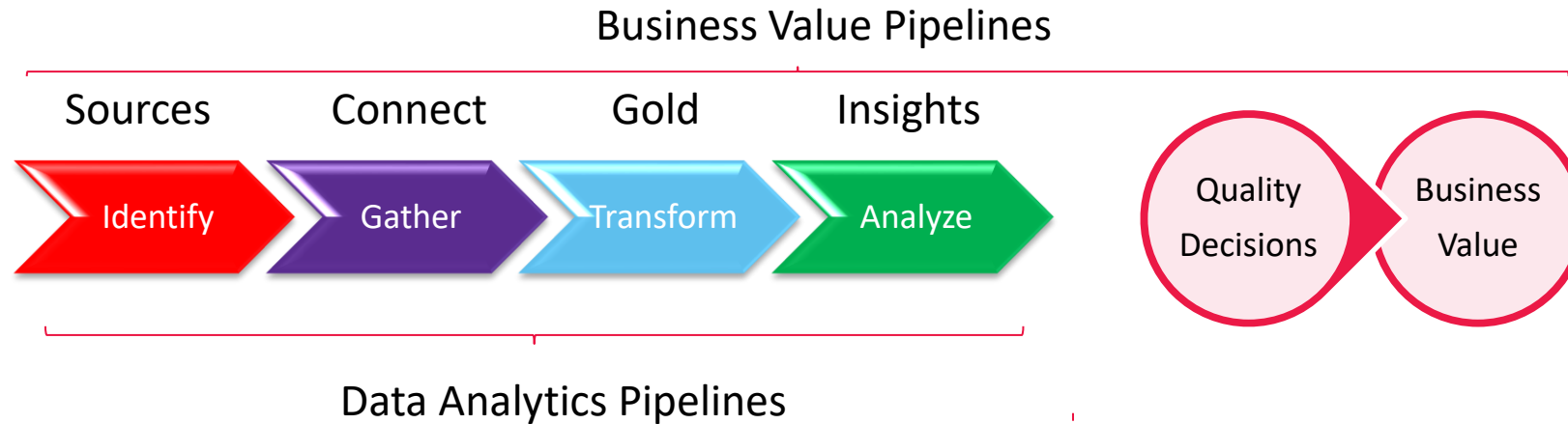
Thanks for joining in, Webinar will commence shortly.



Data Pipeline Observability

Dec 2020

Background of Decision Engineering in Enterprises



Key Trends

- Organization with **High Quality decisions yields 2x of business values** as against the laggards
- Quality decisions are influenced by data analytics pipelines health
- **More than 80% of organizations** achieves highest decision-making score with better focus on data-analytics pipelines health
- Organizations thrive for high yielding pipeline value chain to enable high quality decisions

High Quality Decisions are defined by following characteristics

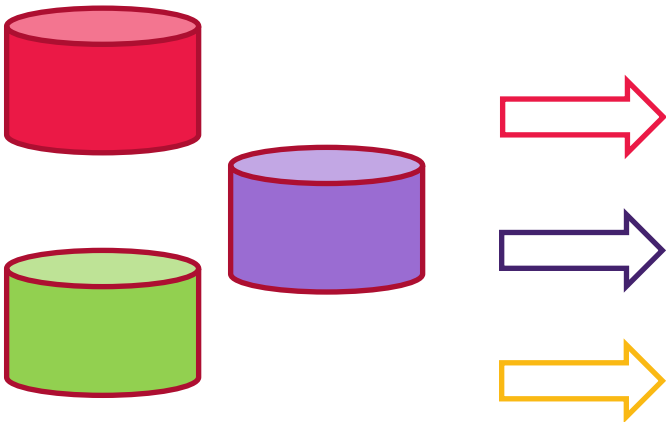
- Invest more time in analysis as against preparation
- Eliminates human biases with the practice of data-driven decisions
- Requires evaluation of the data in the definition phase
- Arrest the leakage of values in the data pipeline journey

Silos are too common

Data Driven Business
Decisions Cycle

Business SLAs

Business Value
Gained/Lost



Fraud, Wastage,
Detection
within 20 mins

Net Margin out of
Schemes/Services

Customer Lead
to target within
within 2 Hours

Lost business
Leads / Customer
mapping

Payment
Reconciliation
within 5 mins

Sales
Opportunities

Realtime
Notification of
incident/accide
nts **within 5
mins**

Accidents impacting
human life,
Violation of regulation
Leading penalties

80% of organizations report
High to Moderate degree of Data Silos

2/3 experience some degree of shadow
(or rogue) data repositories

Close to 70% are unable to provide a
Comprehensive single customer view

Source: Big data insights & opportunities, Comp TIA

Taking significant cost and time to design,
Build and maintain

Rely on Schema dependent data

Batch-only data to Stream

Shortage of skills to build

Numerous vendors, tools leveraged to
develop data pipelines

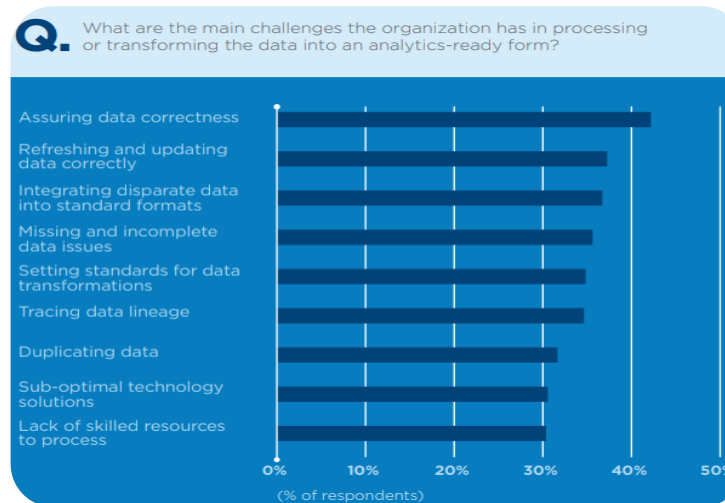
Modern Challenges & Problems of Data Pipelines Value Chains

Sources & Collect Phase



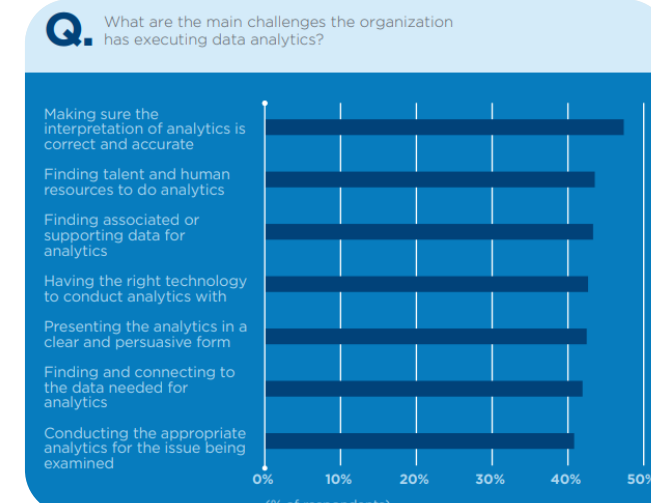
- **Relevant data** capture
- Proper **Data Schemas** / Meta data definitions
- **Timely** collection of **data**
- **Lacks** data **context**

Golden Phase



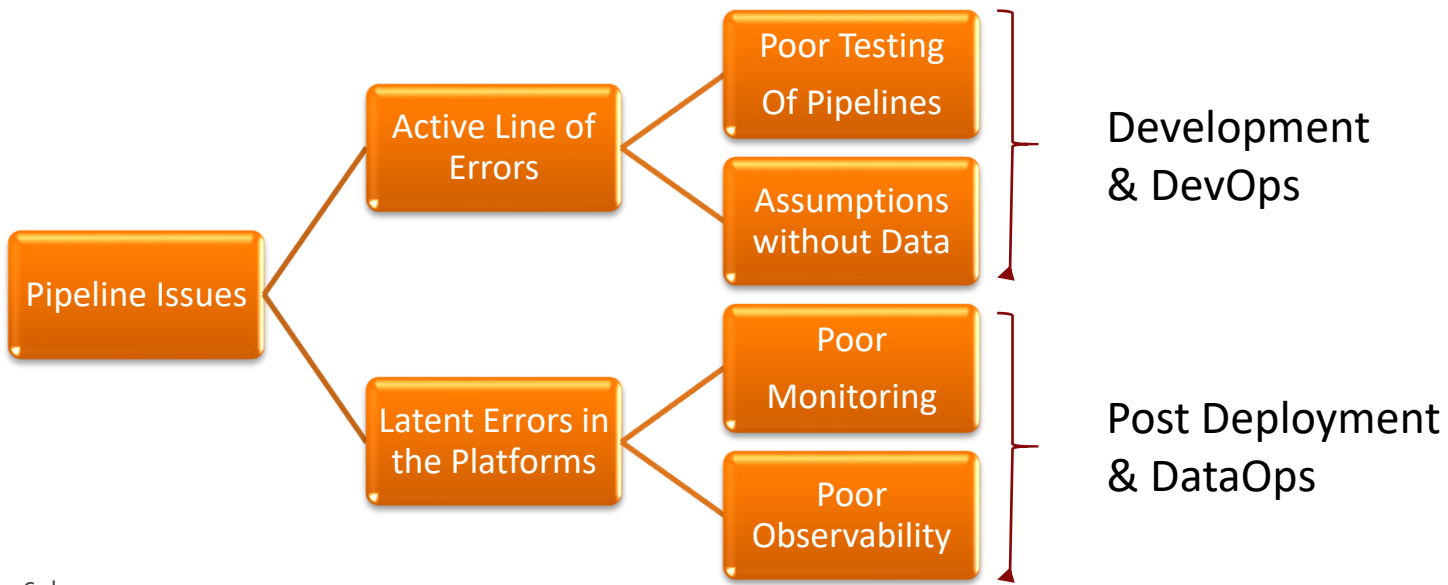
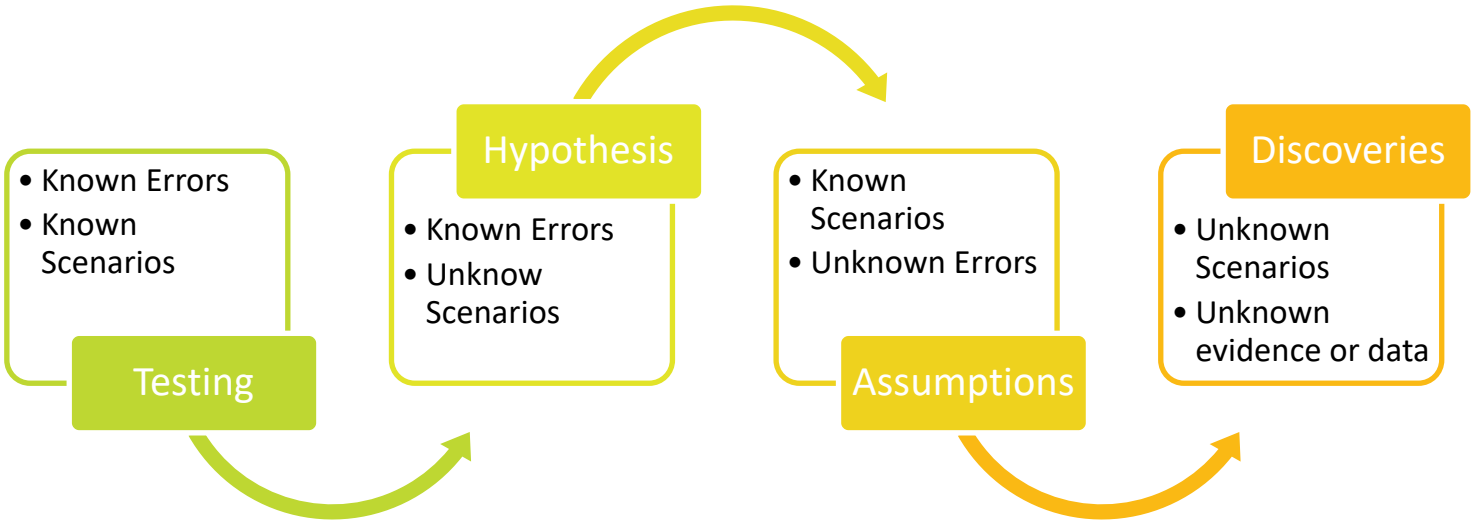
- **Data Quality**
- **Integrate** data with context
- **Trace** data **lineage**
- **Missing** or partial **issues**
- **Lack** of skilled **resources**

Insight Phase



- **Poor Interpretability** of data
- **Lack** data **to evidence**
- **Lack** of right **technologies**
- **Lack** of skilled **resources** to **subjective outcome**
- **Explain ability of Analytics Outcome**

Evolution from Testing to Observability



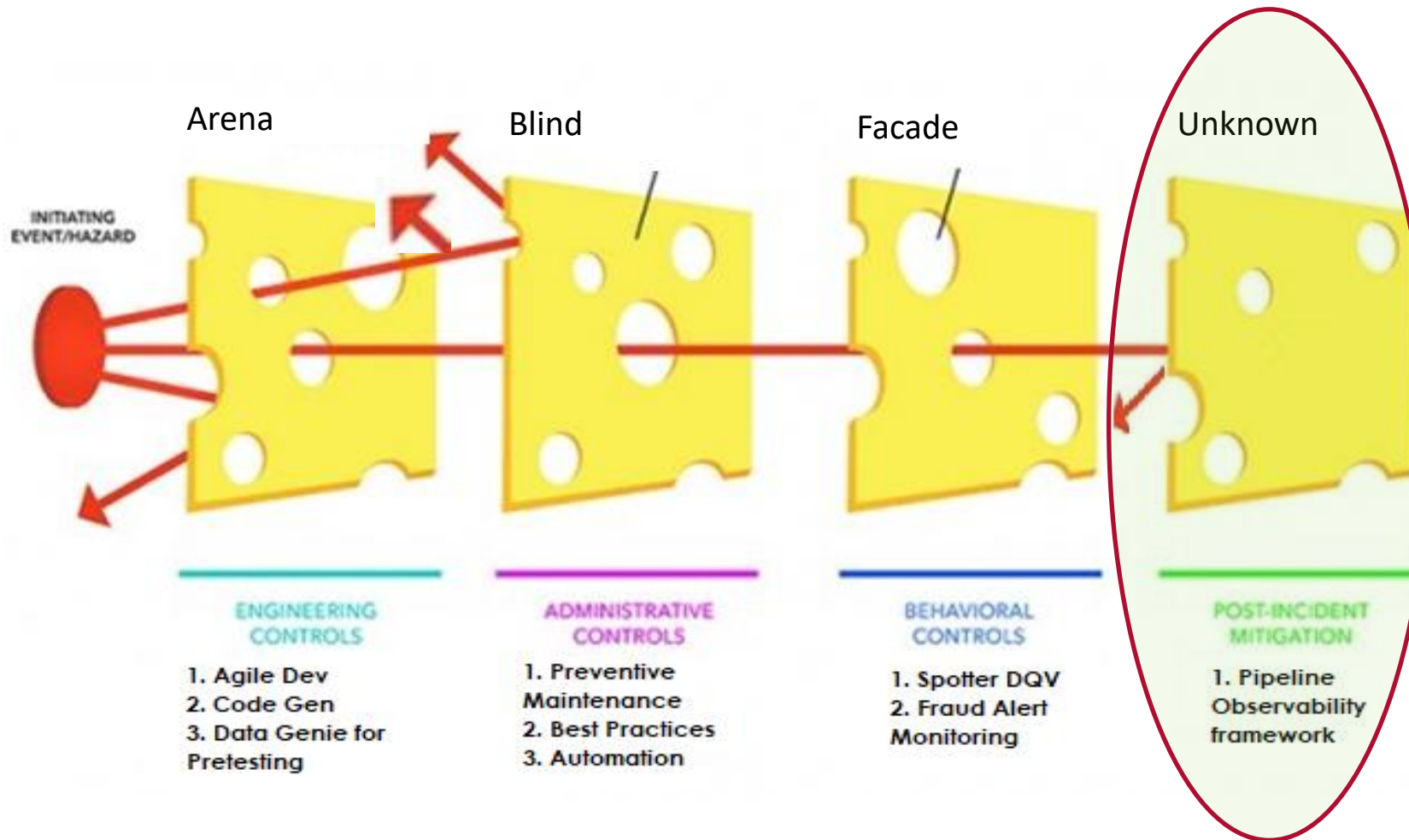
Understanding

Unknown-Known Assumptions Orchestrator	Known-Known Facts Testing
Unknown-Unknown Discoveries Observability	Known-Unknown Hypothesis Monitoring

Data Available

Analogy to Pipeline Failures

The failure fits well into Swiss-Cheese Model



- Why is data pipeline x broken?
- What services does my service depend on – and what services are dependent on my service?
- “What went wrong during this release?”
- “Why has performance degraded over the past quarter?”
- “What is system performance like for our most important customers?”
- What did my service look like at time point x?
- What was the relationship between my service and x at time point y?
- What was the relationship of attributed across the system before we deployed?
- What is most likely contributing to latency right now? What is most likely not?
- Are these performance optimizations on the critical path?

Courtesy : Michigan Tech

Our Experience / Projects within HCL



Fraud Analytics Platform – System stability issue causing payment Red-Alert flagging – **Monthly ~50K USD, 2-3 Hours outage/month**



Analytics Platform – System stability causing Spark job fails, Implicit data issues due to job failures, Null values, Cron job issues
- **Everyday takes about 2-4 hours just for monitoring job failures.**



Product Data Analytics – to capture the customer behavior and latent issues

... and many more where HCL has key stakes in managing modernized digital platforms



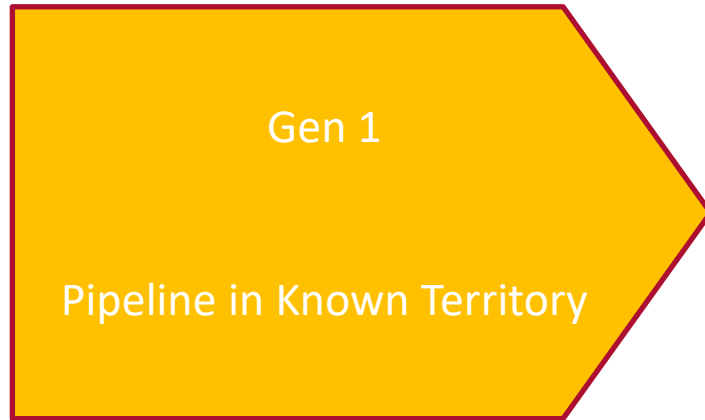
Data Pipeline Observability Use cases & Metrics

Top Use Cases	Key Metrics
Back Pressure	➤ Avg. Ingest Rate per Sec/Consumer Rater per Sec ➤ No. of pending messages over time
Subtask Failures/ Dependencies	➤ Execution time of each sub tasks ➤ Errors of N last failed subtasks ➤ Average/min/max duration of succeeded subtasks ➤ Average/min/max bytes/rows in/out of subtasks ➤ Errors of N last failed subtasks
Delayed Job	➤ Total rows in / Filtered / Unparseable/Out ➤ Total bytes in/out ➤ Fetch Time, Spill Time
Rescheduling Failure	➤ No. of retry count ➤ Duration between Rescheduled task, Original Task
Wrong Configuration	➤ Failed task per Config Error code. ➤ No. of Null Config Errors

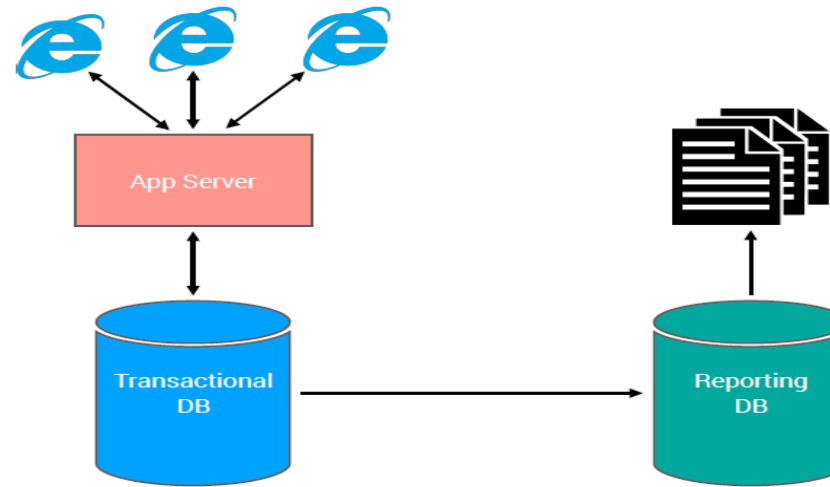
Symptoms to rely on Observability?

- Are you still relying on Customers to notice an outage of Data Pipelines?
- Is your monitoring tools NOT helping to bring down Outage repair cost?
- Is your cost of reactive repairs of Data Pipelines Fixes / Failures too high in terms of missed opportunity?
- Are you still investigating through manual approaches?
- Do the issue backlogs keep piling?
- Is your response strategy to Data Pipeline issues reactive?

How Data Pipelines Architecture changed?



- Monolithic Architecture
- Waterflow Dev. Model
- Architecture rarely changes
- Monitoring system
- Standard list of events indicates the major threshold



Level 1

Traditional Monitoring

Few Long Running Jobs
Server based
Monolithic



Untested Scenarios
Unmonitored Incidents

Reactive, Unknown, Time Consuming, Subjective Issue Resolutions

Nagios
APM
SSH
Debugger
grep
Profiler

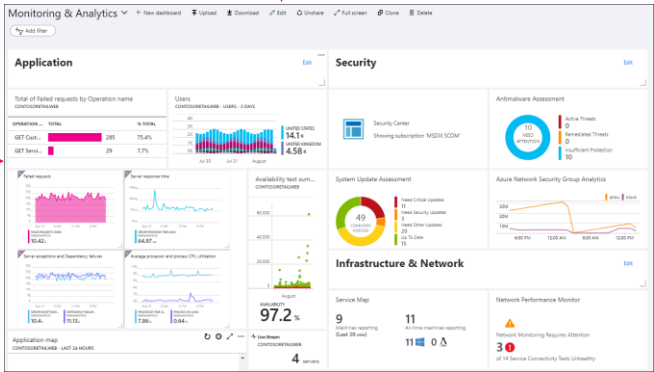


Agents/
Forwarding/
Shipping

Product Development
Support/ Helpdesk
Product Management
Security & Compliance

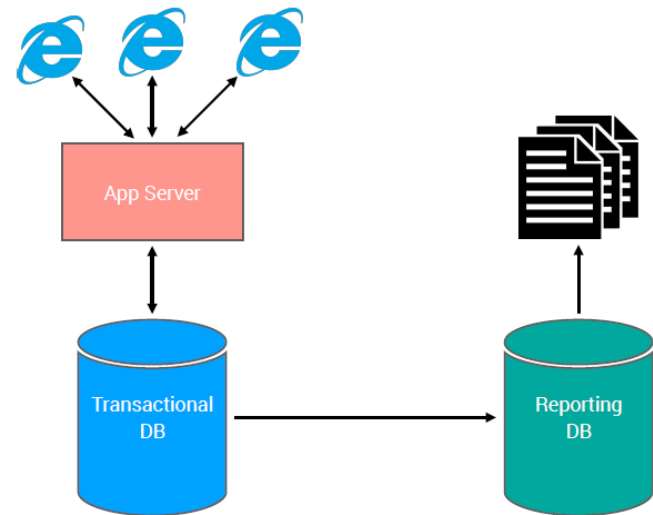


Error Logs
Predefined Metrics
Predefined Events

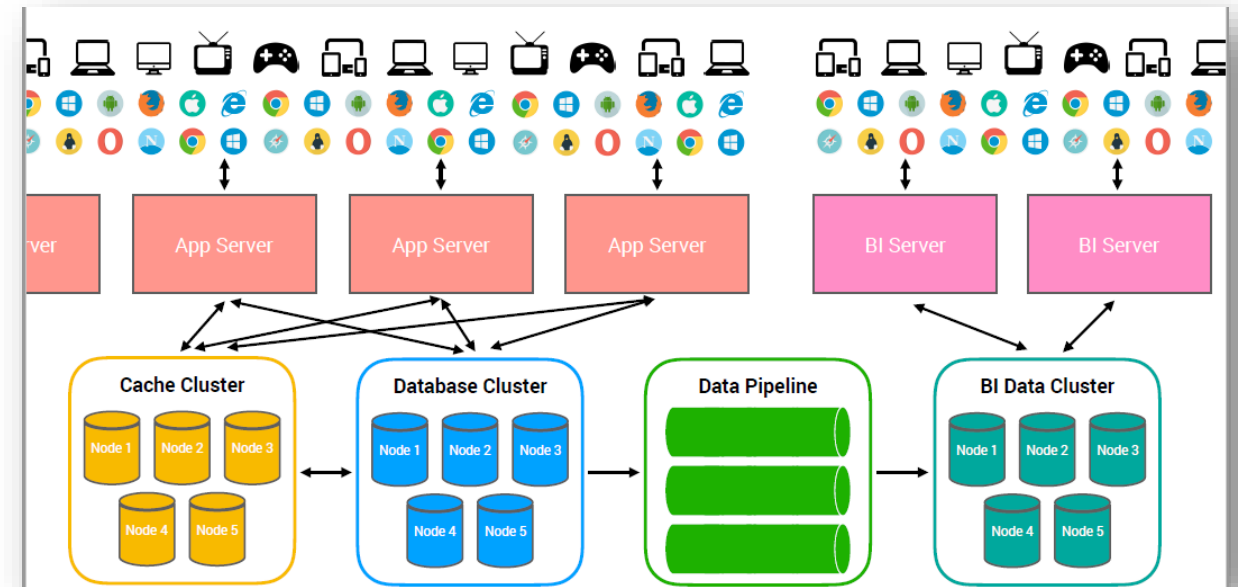


Limited Coverage, Answers What?, Known Scenarios

Journey from Monolithic to Distributed Systems



- Highly Available
 - Multiple Device
 - Faster Response
 - Realtime Analytics
-



How Data Pipelines Architecture Requirements changed?

Pipeline in Known Territory

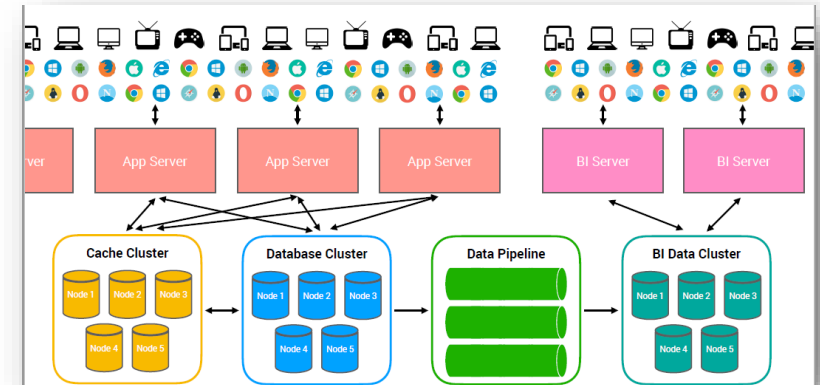
- Monolithic Architecture
- Waterflow Dev. Model
- Architecture rarely changes
- Monitoring system
- Tightly coupled systems
- Standard list of events indicates the major threshold

Level 1

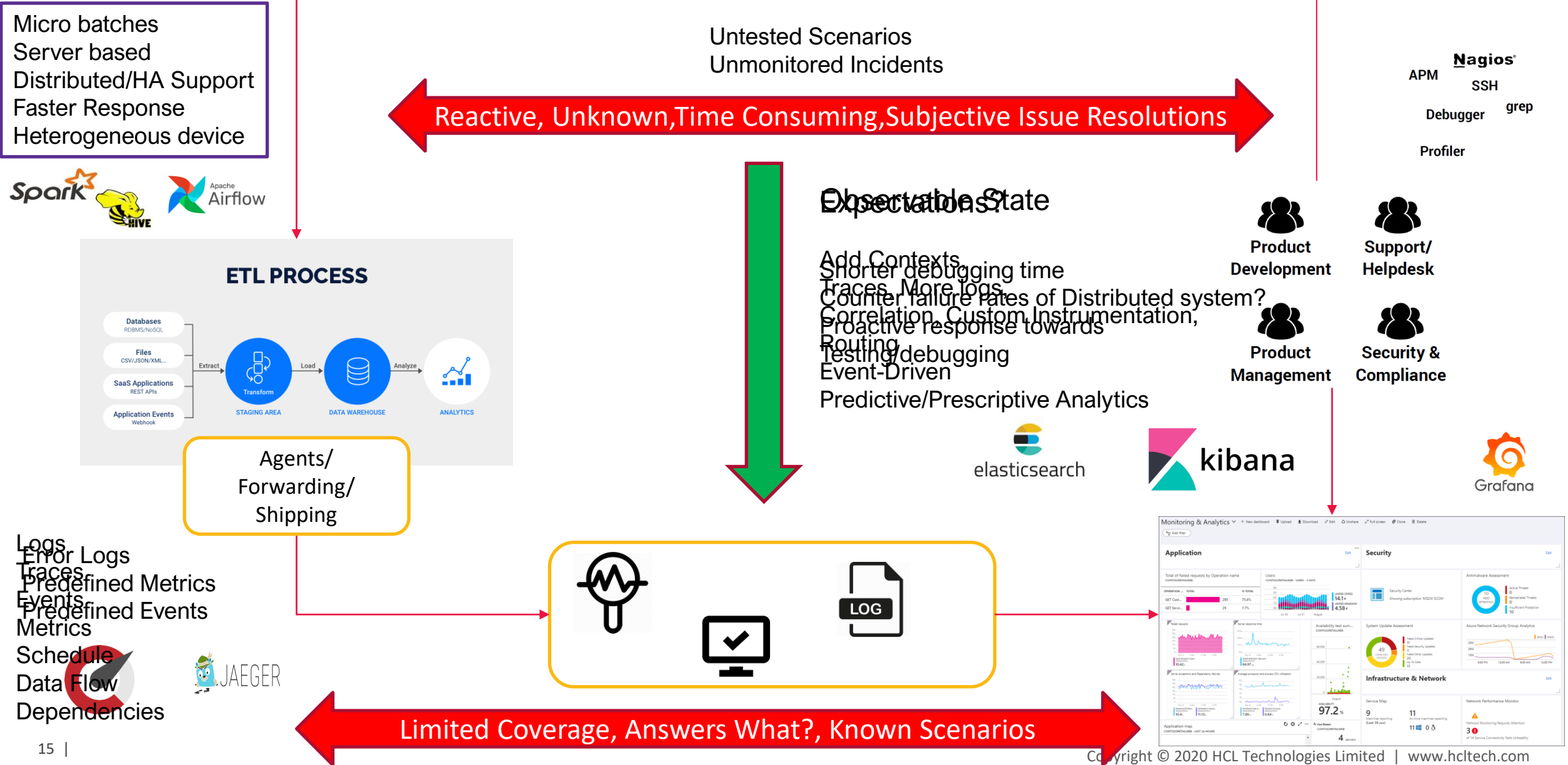
Pipelines in Unknown Territory

- Increased Failures due to distributed systems
- Colocation of pipelines near the data
- Need for proactive measures
- Support needs to answer Why in shorter cycle.
- Preventive/Predictive Maintenance

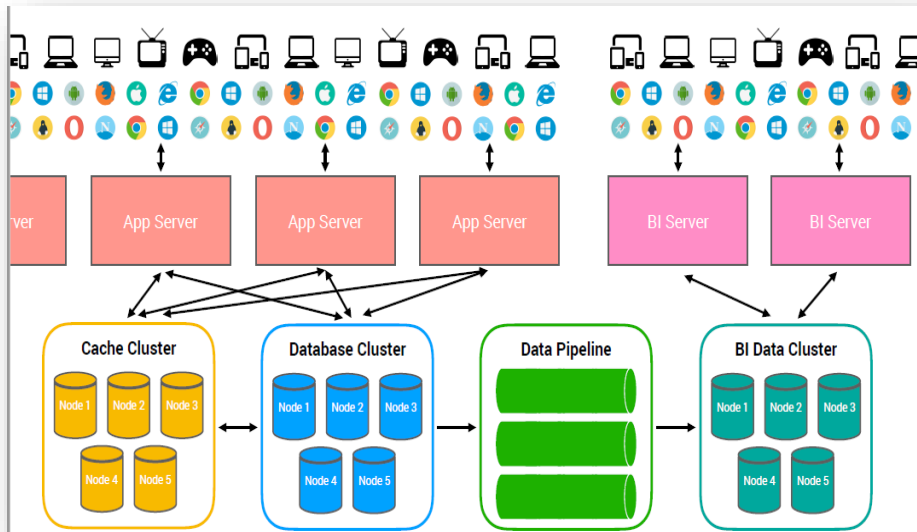
Level 2



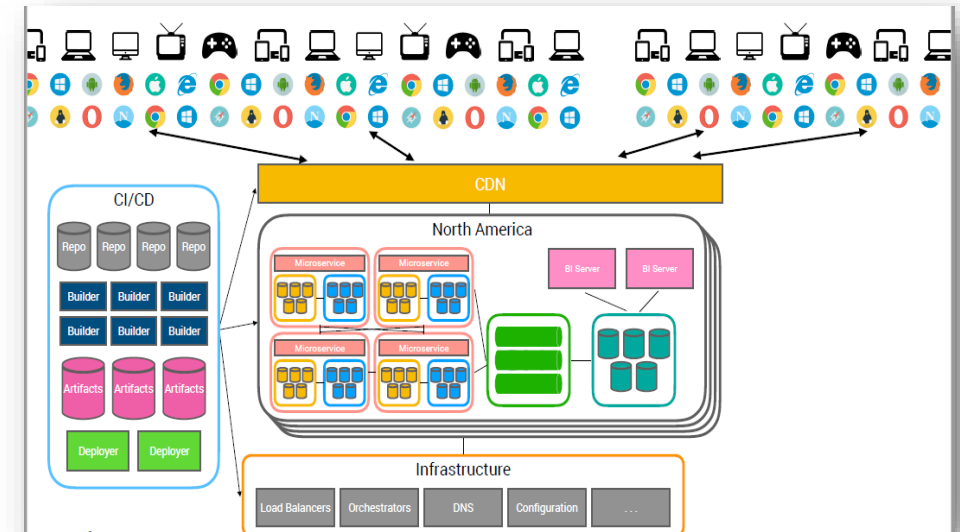
Distributed Monitoring to Observable Systems



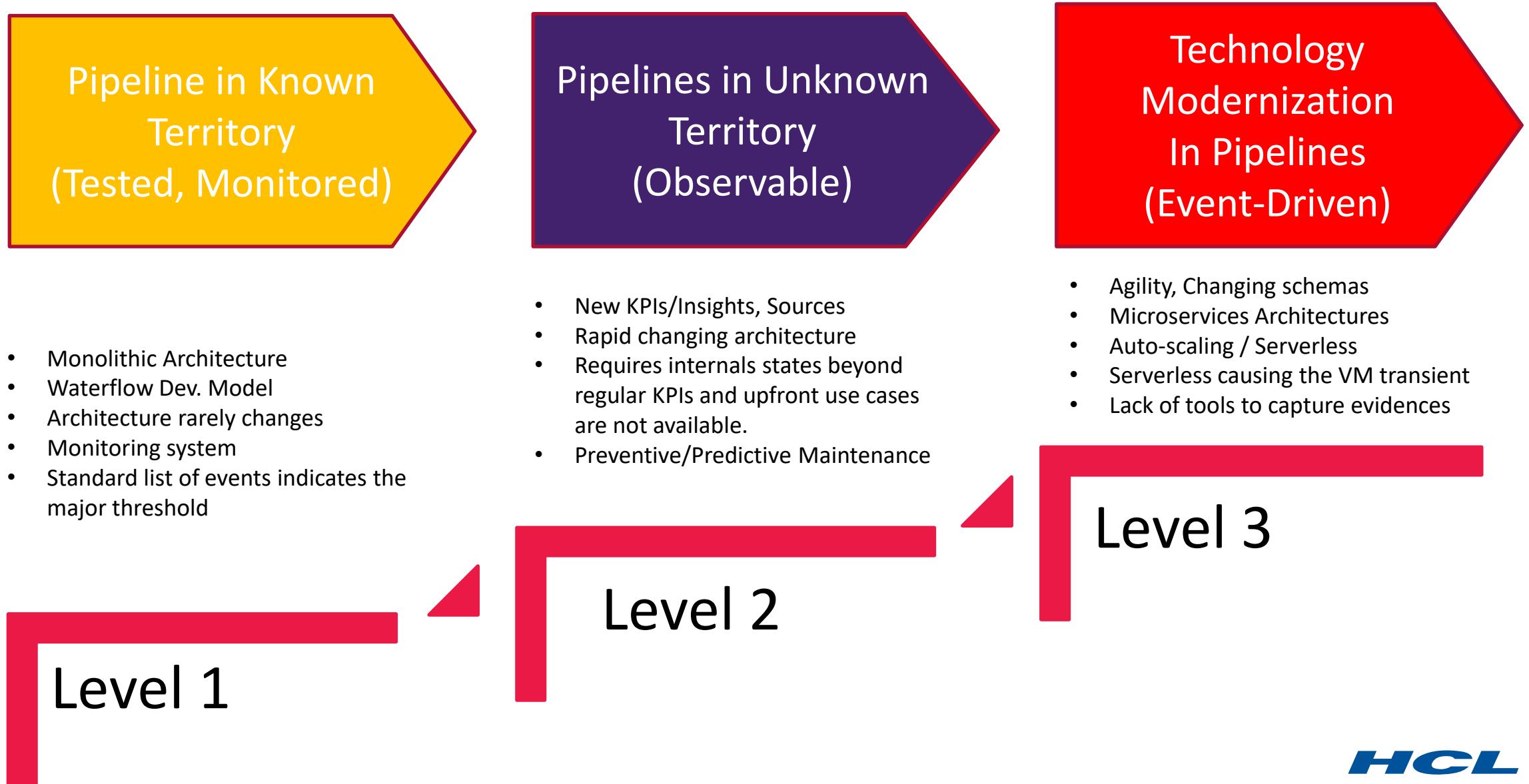
Distributed to Microservices



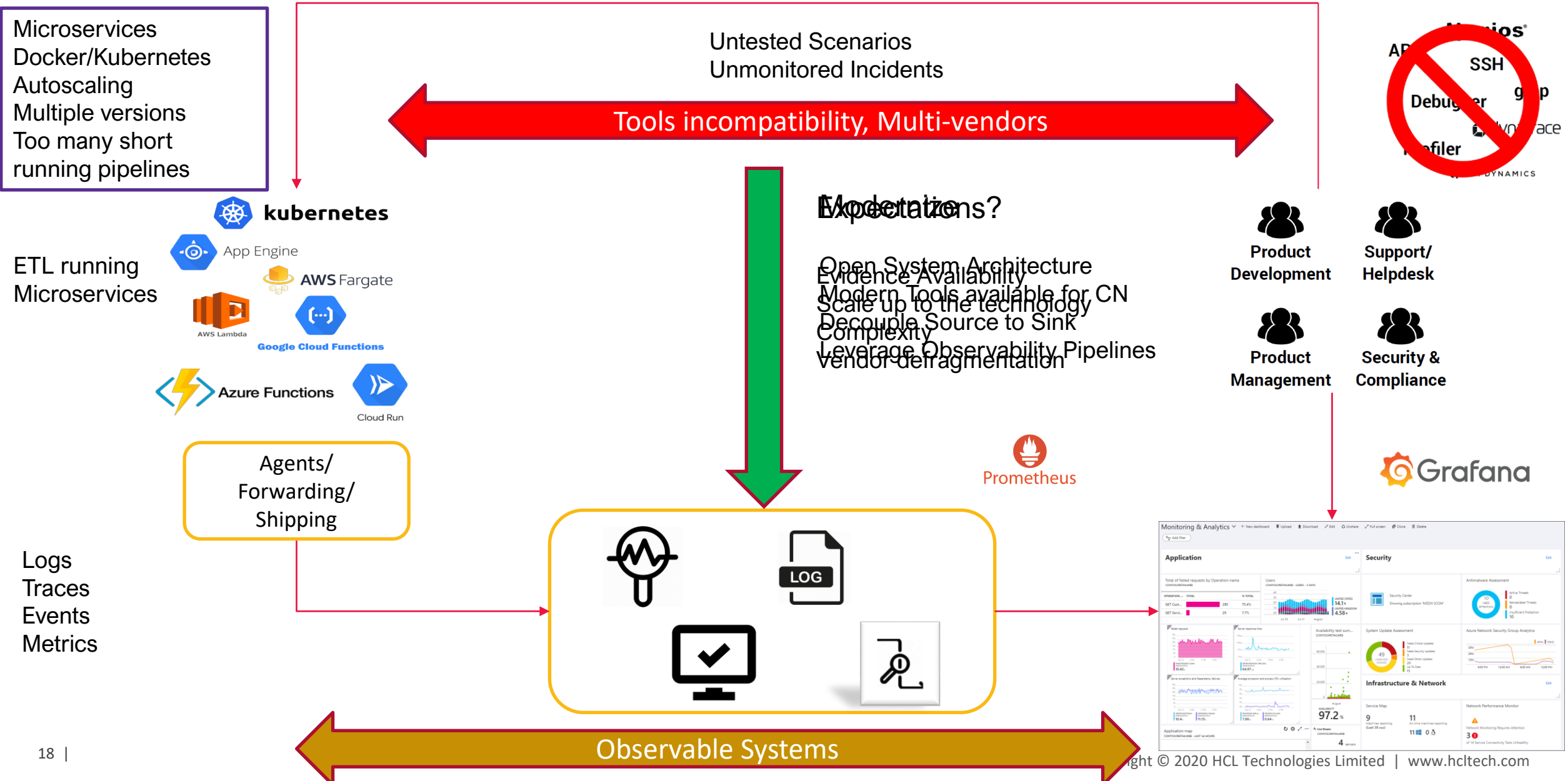
- Multiple release/day
- Multiple Geos
- DevOps Automation



How Data Pipelines Architecture Requirements changed?



Distributed Systems to Microservices



Event Driven Architecture for Observability

Microservices
Docker/Kubernetes
Autoscaling
Multiple versions
Too many short lived
data pipelines

ETL running
Microservices

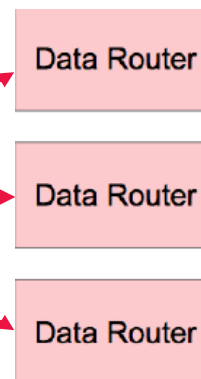
Logs
Traces
Events
Metrics
Schedule
Data Flow
Dependencies



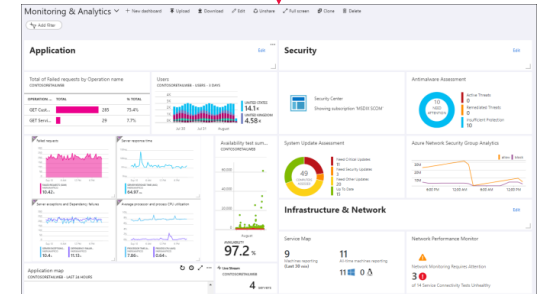
Agents/
Forwarding/
Shipping



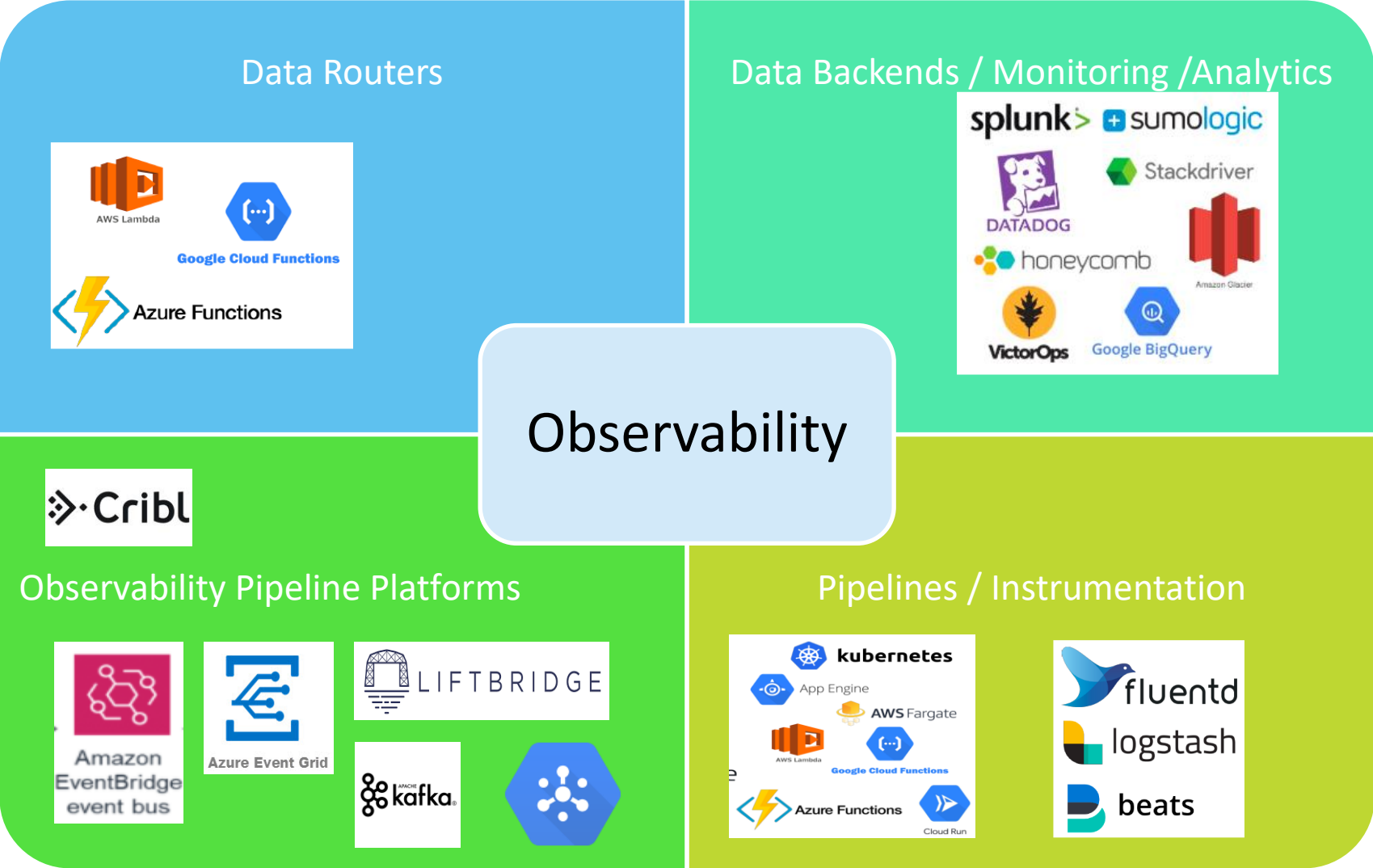
Observability
Pipelines



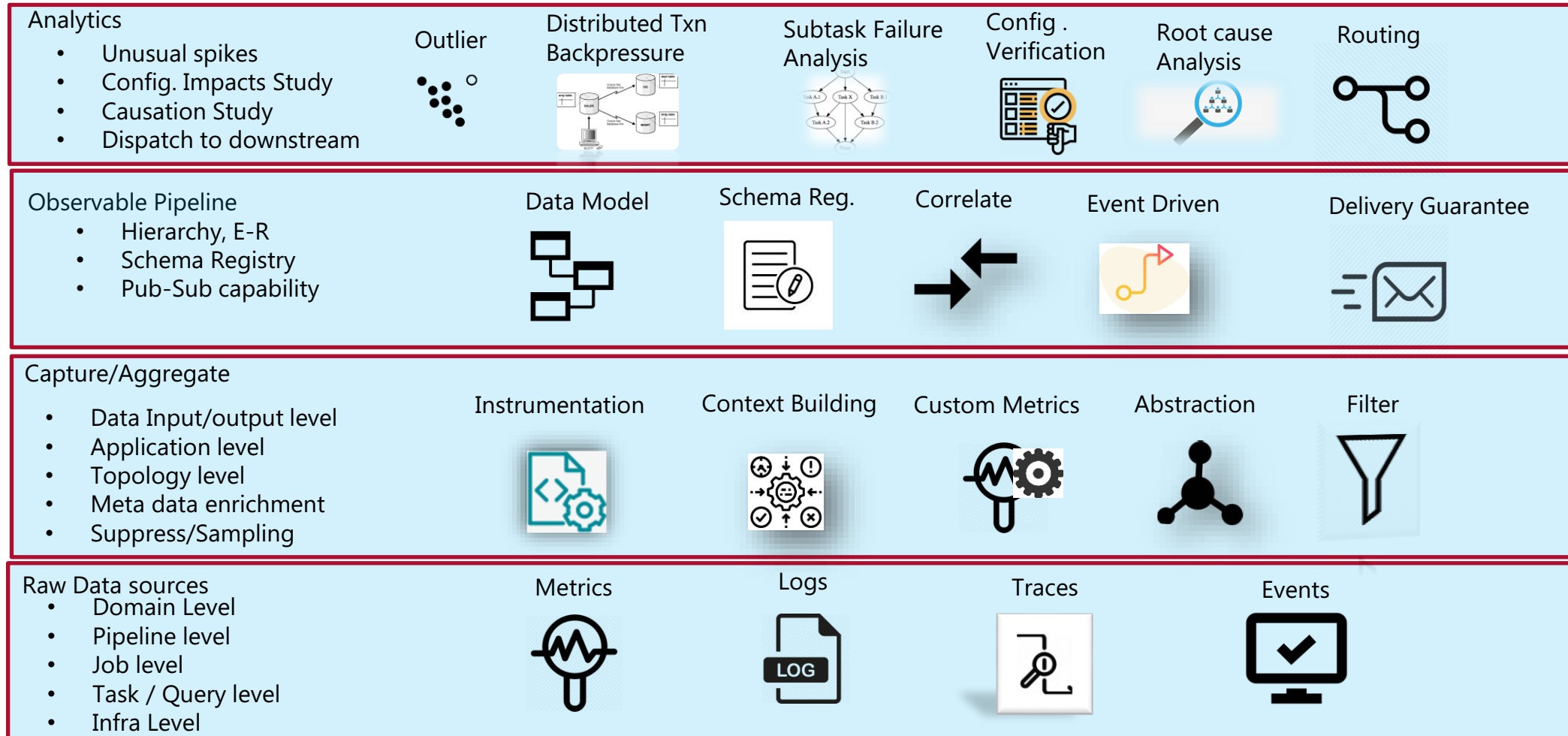
Data
Backend



Market Tools Landscape



Pipeline Observability – Functional View



Data Pipelines
In Production

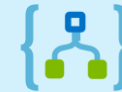


Business Transactions, Microservices Calls, User Interaction, SOA Calls, Batch Jobs,

Pipeline Observability – Technology Mapping

Analytics

- Unusual spikes
- Config. Impacts Study
- Causation Study
- Dispatch to downstream



Observable Pipeline

- Hierarchy, E-R
- Schema Registry
- Pub-Sub capability



Capture/Aggregate

- Data Input/output level
- Application level
- Topology level
- Meta data enrichment
- Suppress/Sampling



Raw Data sources

- Domain Level
- Pipeline level
- Job level
- Task / Query level
- Infra Level



Data Pipelines
In Production

Business Transactions, Microservices Calls, User Interaction, SOA Calls, Batch Jobs,



	Base Observability (Distributed)	Enhanced Observability (Event Driven)
Positive Outcomes	- Pointed and immediate solution for observability - Shorter lead time to observability outcomes - Reduced shutdown/disruption of unplanned/unexplained pipeline failures - Suited for simple data pipelines - Availability of skills - Source pipelines may not require modernization	- Availability of observable data irrespective the source availability - Decoupled architecture gives more flexibility - Flow control of the amount of context/logs/traces to be routed based on applications. - Suited well for complex and growing number of pipelines – eg: Microservices
Negative Implication	- Requires modernization effort of pipelines - Vendor / tool changes may become complex or not feasible. - Extensibility is a challenge when the no. of sources and consumer for observable data. - Productivity overhead - when number of pipelines grows, and becomes complex	- Reduced productivity for less complex pipelines - Management of microservices - Availability of skills in the immediate terms.



made with
Biteable.com

Monday Morning @ ELK IT Services

made with
Biteable.com

Stakeholders Impact

Stakeholders	Customer	Architect & Data Engineer	L2 Support	DevOps
Before Observability	• Erroneous outcomes • Delayed decisions • High budget to repair issues	• Delayed feedback • Reactive response to issues • Little accountability of failures	• Overloaded burden • Lacked evidence during reporting • More escalations from customers	• Lack of visibility of real-world systems • Continuous iteration after the issue reaches customer
After Observability	• Low cost to repair issues • High quality decisions • Encashed opportunities	• Better accountability • Earlier response to issue fix • Increased visibility of pipelines in production	• Reduced workload • Able to substantiate the evidence • Reduced escalations	• Can test for real-world performance • Continuous iteration before the issue reaches customer

Luminous – Observability Integration Platform

F
e
a
t
u
r
e
s

Observability Roadmap

Conduct Pipeline Behavioral Assessment/Planning

Reference Toolchains / Platforms

Coverage of Ref. Architecture On Azure, AWS, Google

Design / Deploy Observability

Open source based deploy, Spotter DQV for Data Quality

Coverage of Microservices deployment

Covers pipeline deployments in Cloud Native environment

Support for Big data Pipelines, Ops Dashboards

HCL Luminous – A system integration platform for DIY enablement of End to End Observability Needs

Consulting/ Delivery / Support/ Built-in Integration / OSS Capability

Luminous is a self-service DIY enablement for Engineering teams for delivering high yielding Data Analytics Pipelines with following key capabilities / use cases

- Backpressures / Subtask failures / Delayed Jobs
- Data Quality Checks
- Lineage of issues tracing back to root cause of failures
- Predictive capabilities to detect down time

Technology Covered

Cloud : Microsoft Azure, Amazon Web Services, Google

OSS - Spark, Logstash, Kibana, Elasticsearch, Fluentd, Jaeger

Monitoring /Visualization : Grafana,

Analytics : Outliers(Dbscan, Kmeans, Autoencoders), Root Cause(Decision Tree),

Preventive Main (RF, LSTM, XGBoost), Pattern Min(CART,SVM,KNN), Correlation

(ARM, Apriori)

Faster Adoption

Enables acceleration of Pipeline Observability by 2X

Improve Pipeline Health

Reduction of Pipeline Errors, and Time to Repair Issues by 30%

Enable DIY (Build & Own Culture)

Shorter deployment due to Pre-built Framework

End-End integration solution for Observability

Right from Instrumentation to Visibility /

Backed Up by AI/ML Algorithms

Supplemented by AI/ML algorithm for Predictive Maintenance, Preventive Maintenance

B
e
n
e
f
i
t
s



HCL Reference Solution Roadmap

Base Version of Observability

- OSS Implementation
- Python based Pipelines

December

- SOP Implementation
- Wrong Configuration
- Backpressure
- Rescheduling failures

February - MVP Stage

- Pub-Sub Implementation
- AWS, Azure, OSS Coverage

November

- Delayed Job
- Partial Job Failures
- Microservices inclusion
- Azure based Implementation

January

- SOP Implementation
- Azure, AWS, OSS Coverage
- Pub-sub implementation

References

1. Observability Pipelines

<https://www.slideshare.net/TylerTreat/the-observability-pipeline>

2. Microservices Observability

<https://www.infoq.com/articles/tyler-treat-observability/>

3. Observability @ Data Pipelines

<https://quickbooks-engineering.intuit.com/observability-data-pipelines-99eda62b1704#:~:text=Observability%20focusses%20on%20providing%20highly,systems%20along%20with%20rich%20context.&text=The%20three%20phases%20of%20the,to%20output%20dashboards%20and%20models>

Feedback



<https://www.surveymonkey.com/r/Dataobs>



<https://excelerator.hcl.com/>>NEXT.ai>>Webinars>



jayachandran.ki@hcl.com

* 1. SAP Id

* 2. Please rate the POV webinar

* 3. Did you gain knowledge about enabling observability for Data Pipelines?

☐ Yes

☐ No

* 4. Would you like to engage NEXT.ai COE for a discussion with your customer

☐ Yes

☐ No

5. Is there a specific topic on which you would want an SME to present a POV or TechRadar that would be of interest to your customer? The topic can be in any technology domain including analytics, big data, cloud, devops, microservices, design, etc.

Done

